Système d'exploitation : TD8

mars 2026 — Pierre Rousselin

Exercice 1 : Opérations bit-à-bit

On considère une variable

```
uint32_t n;
```

qu'on suppose initialisée. Le type `uint32_t` est un type entier non signé toujours sur 32 bits.

En utilisant uniquement les opérateurs de *décalage à droite* `>>`, de *décalage à gauche* `<<`, *et* bit-à-bit `&`, *ou* bit-à-bit `|`, *ou exclusif* bit-à-bit `^`, *négation* bit-à-bit `~` et des constantes entières (écrites en décimal ou hexadécimal), donner les bouts de code pour

1. mettre dans une variable `m` le produit de `n` par `4` ;
2. mettre dans des variables `q` et `r` le quotient et le reste de la division euclidienne de `n` par 2 ;
3. mettre dans des variables `q` et `r` le quotient et le reste de la division euclidienne de `n` par 16 ;
4. mettre dans une variable `oct` de type `uint8_t` le contenu de l'octet de poids fort de `n` ;
5. mettre dans une variable `oct` de type `uint8_t` le contenu du deuxième octet le plus fort de `n` ;
6. mettre dans une variable `b` de type `uint8_t` la valeur du bit numéro 0 (c'est-à-dire son bit de poids le plus fort) de `n` ;
7. écrire tous les bits de `n` en commançant par celui de poids le plus fort ;
8. mettre dans une variable `rot` la rotation de `n`, c'est-à-dire la valeur obtenue en échangeant les deux derniers octets avec les deux premiers.

--- * ---

1 Boutisme et conversion

On aura besoin pour la partie réseau de fonctions capables de convertir des entiers en gros-boutiste.

Exercice 2 : fonctions de conversion

Définir les fonctions suivantes

```
uint32_t machine_to_big32(uint32_t machine_n);
uint32_t machine_to_little32(uint32_t machine_n);
uint32_t big_to_machine32(uint32_t big_n);
uint32_t little_to_machine32(uint32_t little_n);
```

sur une machine *little-endian* ; sur une machine big-endian (dire ce qui change). On pourra utiliser les opérations bit à bit ou bien utiliser des conversions de pointeurs.

--- * ---